INFORME TÉCNICO Y ARQUITECTÓNICO INTEGRAL

Plataforma de Streaming OTT Multimedia — **PopConly**

Auditoría Completa: Tecnologías, Código, Persistencia Políglota, UI/UX, Pagos y DevOps

Versión del Sistema: 0.1.0 | Next.js 16.3.8 (App Router) + React 19.2.8 | TypeScript 5
Modelo de Persistencia: Políglota Distribuido (PostgreSQL 16, MongoDB 7, Redis 7, MySQL 8.4)
Infraestructura de Despliegue: Docker Multi-stage + Oracle Cloud (OCI Ampere A1 ARM64) + Caddy SSL
Pasarelas & Cobros: Wompi API (Tokenización PCI) + Conciliación Nequi + Panel Administrativo
Fecha de Emisión: Octubre 2026 | Documento de Especificación y Auditoría Técnica

PARÁMETRO	DETALLE ESPECÍFICO DEL SISTEMA
Nombre del Producto	PopConly (Plataforma SVOD / OTT Multimedia)
Framework Principal	Next.js 16.3.8 (App Router, Server Actions, Standalone Build)
Librería de Vista	React 19.2.8 + React DOM 19.2.8 (Concurrencia, Server Components)
Lenguaje Base	TypeScript 5 (Tipado Estricto de Extremo a Extremo)
Motor de Estilos	Tailwind CSS v4 (@tailwindcss/postcss) con Tokens HSL/HEX Personalizados
Bases de Datos (4)	PostgreSQL 16 (Auth) | MongoDB 7 (Catálogo) | Redis 7 (Caché) | MySQL 8.4 (Cobros)
Seguridad & Autenticación	Auth.js v5 (NextAuth beta 32), Bcryptjs (12 rondas), Nodemailer OTP/Magic-Link
Pasarela Financiera	Wompi API v1 (Tokenización PCI) + Comprobantes Manuales Nequi + Panel Admin
Contenerización	Docker 22-alpine Multi-stage + Docker Compose v2 (5 servicios interconectados)
Despliegue Objetivo	Oracle Cloud Infrastructure (OCI Always Free: 4 OCPU ARM64, 24GB RAM)

1. RESUMEN EJECUTIVO Y VISIÓN GENERAL

PopConly es una solución integral de plataforma de streaming de vídeo bajo demanda (SVOD/OTT) diseñada con estándares de ingeniería de software modernos de alta gama. Su arquitectura emula los patrones de líderes de la industria como Netflix, Disney+ y Prime Video, combinando una experiencia visual cinematográfica, navegación fluida de contenidos 4K HDR, un reproductor interactivo con seguimiento en vivo de línea temporal, y un motor de persistencia políglota distribuido en cuatro bases de datos especializadas.

Una de las fortalezas más notorias de su ingeniería es el **patrón de degradación elegante (Graceful Degradation)**: el sistema está completamente cableado para producción con Docker, PostgreSQL, MongoDB, Redis, MySQL y pasarelas bancarias, pero a su vez posee capas de contingencia que le permiten ejecutarse en entornos locales sin servidores externos activos, utilizando catálogo estático en memoria y almacenamiento local (LocalStorage) sin provocar caídas ni errores.

2. MATRIZ DETALLADA DE TECNOLOGÍAS Y DEPENDENCIAS

A continuación se presenta el inventario completo de dependencias de producción, herramientas de desarrollo y tecnologías analizadas en los archivos package.json, docker-compose.yml y Dockerfile:

TECNOLOGÍA	VERSIÓN	CAPA	JUSTIFICACIÓN TÉCNICA EN POPCONLY
Next.js	`16.3.8`	Fullstack Core	App Router con Server Components para SEO y velocidad; Server Actions para mutaciones RPC; empaquetado standalone para imágenes Docker ultraligeras.
React / DOM	`19.2.8`	Frontend UI	Librería reactiva para UI dinámica, renderizado concurrente, hooks reactivos avanzados y manipulación fluida de medios.
TypeScript	`^5.0.0`	Lenguaje	Tipado estricto extremo a extremo garantizando consistencia en entidades de catálogo, suscripciones, perfiles y payloads de pago.
Tailwind CSS	`^4.0.0`	Estilos / UI	Motor CSS v4 con directiva @theme, sin hojas de estilos pesadas ad-hoc, aceleración GPU para micro-animaciones cinematográficas.
Prisma ORM	`^6.19.3`	ORM / Postgres	Mapeo relacional de PostgreSQL con migraciones versionadas (prisma/migrations), tipado estricto autogenerado y pooling de conexiones.
NextAuth (Auth.js)	`^5.0.0-beta.32`	Seguridad Auth	Autenticación universal basada en JWT, soporte dual de credenciales y magic links, integración nativa con middleware de protección.
@auth/prisma-adapter	`^2.11.3`	Adaptador Auth	Sincroniza y almacena de forma estandarizada los modelos User, Account, Session y VerificationToken en PostgreSQL.
bcryptjs	`^3.0.3`	Criptografía	Hashing robusto con sal y 12 rondas para contraseñas de cuentas, además de validación de tokens OTP temporales de 6 dígitos.
mongodb (Driver)	`^7.7.0`	Driver NoSQL	Conexión oficial de alto rendimiento a MongoDB para consultar títulos, sinopsis, géneros, slugs y URLs de streaming de vídeo.
ioredis	`^6.0.0`	Caché / K-V Store	Cliente Redis de alta concurrencia: gestiona minuto de reproducción, favoritos y motor de recomendaciones sin golpear discos SQL.
mysql2	`^3.24.5`	Cliente SQL	Manejador con pool de promesas para MySQL 8.4: persistencia del ledger transaccional de Wompi, planes y auditoría financiera.
nodemailer	`^7.0.13`	Email Transaccional	Envío de correos de login con magic links y códigos OTP de acceso (vía SMTP Brevo/OCI en producción o consola en local).
tsx	`^4.23.15`	CLI / Tooling	Ejecución directa de scripts TypeScript sin paso previo de compilación (usado en scripts/seed-catalog.ts para siembra de MongoDB).
Docker & Compose	`v2 / Node 22`	DevOps / Infra	Contenerización multi-servicio con soporte nativo ARM64 para la máquina Ampere A1 Always Free de Oracle Cloud.

[RENDIMIENTO EXTREMO] Alineación con la Filosofía de Rendimiento Moderno

El proyecto no utiliza librerías de UI innecesariamente pesadas (como Material UI o Ant Design) que degradan la velocidad de carga (LCP/FID). En su lugar, utiliza componentes nativos en Tailwind CSS v4, lo cual garantiza un First Contentful Paint (FCP) inferior a 0.8 segundos y una experiencia táctil ultra reactiva.

3. ARQUITECTURA DE PERSISTENCIA POLÍGLOTA (LAS 4 BASES DE DATOS)

PopConly utiliza un enfoque de ingeniería de datos avanzado denominado **Polyglot Persistence**, donde cada motor de almacenamiento se selecciona específicamente según las características de la carga de trabajo:

MÓDULO 1: PostgreSQL 16 (Relacional ACID) **Identidad, Cuentas, Perfiles y Sesiones**	**MÓDULO 2: MongoDB 7 (Documental NoSQL)** **Catálogo Multimedia, Metadatos y Búsqueda**
• **Manejado por:** Prisma ORM (^6.19.3). • **Tablas principales:** User, Account, Session, VerificationToken, Profile. • **Propósito:** Consistencia ACID rígida para gestión de identidad, tokens JWT, códigos OTP temporales y múltiples perfiles de visualización (ej. Elena, Alex Kids). • **Seguridad:** Hashing bcrypt 12 rondas para contraseñas de acceso.	• **Manejado por:** Driver nativo mongodb (^7.7.0). • **Colección:** titles (con índices en slug único y texto completo). • **Propósito:** Esquema NoSQL flexible para almacenar películas, series, géneros, sinopsis, metadatos 4K/Dolby Vision, URLs HLS de vídeo y repartos. • **Ventaja:** Consultas JSON directas y siembra automatizada via tsx.
MÓDULO 3: Redis 7 (In-Memory K-V) **Capa de Ultra-Velocidad y Estado Temporal**	**MÓDULO 4: MySQL 8.4 (Relacional Transaccional)** **Suscripciones, Facturación y Auditoría Wompi**
• **Manejado por:** ioredis (^6.0.0). • **Estructuras:** Hash para progreso (popconly:progress:{id}), Set para favoritos (popconly:fav:{id}). • **Propósito:** Guardar el minuto exacto de reproducción cada pocos segundos sin sobrecargar discos relacionales. Alimenta la sección 'Continuar viendo' y el motor algorítmico de recomendaciones.	• **Manejado por:** mysql2/promise (^3.24.5). • **Tablas:** plans, subscriptions, payments (db/billing.sql). • **Propósito:** Transaccionalidad financiera estricta para cobros con Wompi, registro de pagos manuales por Nequi, estados de membresía y fechas de corte.

[ARQUITECTURA CLAVE] Mecanismo de Desacoplamiento y Resiliencia Local

Cada archivo de conexión en **lib/** (lib/prisma.ts, lib/catalogDb.ts, lib/redis.ts, lib/mysql.ts) implementa bloques de captura de excepciones y verificación de cadenas de entorno. Si una base de datos no está disponible, la aplicación **no se rompe**: el catálogo recurre al archivo en memoria lib/data.ts, el progreso se almacena en LocalStorage del navegador y la facturación opera en 'modo visual'. Esto permite desarrollar y presentar demos sin dependencias activas.

4. DESGLOSE DEL CÓDIGO FUENTE (VISTAS Y RUTAS FRONTEND)

La capa de presentación aprovecha las capacidades de Next.js App Router combinando páginas de cliente (para interactividad multimedia inmediata) y componentes de servidor (para rendimiento SEO y metadatos dinámicos):

RUTA / VISTA	TIPO	DESCRIPCIÓN DE LA EXPERIENCIA Y LÓGICA DE NEGOCIO
`app/page.tsx`	Client Page	Landing comercial de alta conversión: Hero con grid dinámico de pósteres en fondo, formulario con validación asíncrona de correo mediante Server Action (checkUser), comparador de planes y sección FAQ interactiva.
`app/browse/page.tsx`	Client Page	Dashboard del suscriptor: Billboard Hero con reproducción de ambientación y video glow, carrusel 'Continuar viendo' alimentado en tiempo real por Redis, carrusel Top 5 y filtros rápidos por género.
`app/watch/page.tsx`	Client Page	Reproductor de vídeo cinematográfico customizado: barra de scrubber interactiva, HUD retráctil tras inactividad del cursor, atajos ±10s, alternador de mute/volumen y guardado periódico de segundo exacto a /api/progress.
`app/titulo/[slug]`	Server Comp.	Ficha SEO individual para cada película o serie. Inyecta JSON-LD oficial (Schema Movie/TVSeries), títulos canónicos dinámicos y algoritmo de títulos relacionados por afinidad de género.
`app/login/page.tsx`	Client Page	Autenticación bifásica: detecta si la cuenta ya existe; si existe, genera un código OTP de 6 dígitos al correo; si no existe, envía un magic-link para registro y configuración de contraseña.
`app/signup/*`	App Flow	Embudo de conversión multi-paso: selección de plan (Essential, Plus, Ultra), asignación de contraseña y pasarela de pago (/signup/payment).
`app/signup/payment`	Client Page	Checkout híbrido: tokenización de tarjetas con Wompi en cliente (cero exposición de datos sensibles) o reporte de transferencia Nequi con comprobante.
`app/admin/subscriptions`	Client Page	Consola de administración protegida por cabecera x-admin-key: visualiza transferencias manuales en cola y permite aprobar o rechazar con un clic activando la membresía en MySQL.
`app/my-list/page.tsx`	Client Page	Gestor de contenido guardado y descargado. Sincroniza estados bidireccionales entre LocalStorage y Redis (/api/favorites) con pestañas Todos, Películas, Series y Descargados.
`app/profile/page.tsx`	Client Page	Panel de perfiles de reproducción (adulto/infantil), control parental, gestión de dispositivos vinculados y preferencias de streaming.
`app/categories`	Client Page	Explorador temático de géneros (Acción, Anime, Sci-Fi, Terror, etc.) con filtros dinámicos y presentación en cuadrícula.
`app/movies & series`	Client Pages	Vistas dedicadas para navegación exclusiva de largometrajes o series maratoneables con banners destacados.
`proxy.ts`	Auth Guard	Middleware que intercepta rutas protegidas (/browse, /watch, /profile, etc.) redirigiendo a usuarios no autenticados al login.

5. DESGLOSE DEL CÓDIGO FUENTE (CAPA SERVIDOR Y ROUTE HANDLERS)

La carpeta app/api aloja los servicios RESTful del servidor, complementados por Server Actions en lib/actions.ts:

ENDPOINT / ACCIÓN	MÉTODO	INTEGRACIÓN	DESCRIPCIÓN DEL PROCESAMIENTO TÉCNICO
`app/api/auth/[...nextauth]`	GET/POST	Auth.js + Postgres	Controlador central de sesiones, firma y validación de tokens JWT, callbacks de sesión y orquestación de proveedores.
`app/api/checkout`	POST	Wompi API + MySQL	Crea la suscripción en estado 'pending' en MySQL, formaliza el cobro del token con Wompi y almacena el registro en 'payments'.
`app/api/webhooks/wompi`	POST	Cripto Webhook	Receptor de eventos de Wompi. Valida checksum SHA-256 usando WOMPI_EVENTS_SECRET y activa la membresía (+30 días) en MySQL.
`app/api/payments/manual`	POST	Nequi + MySQL	Registra comprobantes de transferencias manuales para revisión en el panel administrativo.
`app/api/progress`	GET/PUT	Redis Hash	Lee y escribe el segundo/minuto exacto en 'popconly:progress:{userId}' con expiración LRU de 365 días.
`app/api/favorites`	GET/POST	Redis Set	Añade o retira identificadores de títulos en el conjunto de favoritos del usuario en Redis ('popconly:fav:{userId}').
`app/api/recommendations`	GET	Redis + Catálogo	Calcula el género dominante entre el contenido visto y devuelve hasta 8 títulos recomendados no consumidos aún.
`app/api/me/subscription`	GET	MySQL Billing	Devuelve el estado de la membresía (activa/vencida) y fecha de corte para autorizar la reproducción del catálogo.
`app/api/admin/payments`	GET/POST	Admin Security	Lista pagos pendientes y ejecuta su resolución manual (approve/reject). Protegido mediante la cabecera x-admin-key.
`lib/actions.ts`	Server Actions	Prisma + Bcrypt	checkUser (comprueba existencia en Postgres), registerUser (crea usuario y perfil) y requestLoginCode (genera OTP al correo).

5.1. Módulos Auxiliares de la Capa de Lógica (Directorio lib/)

- **lib/auth.ts:** Configura Auth.js con adaptador Prisma, estrategia JWT y callbacks de usuario.
- **lib/billing.ts:** Abstracciones SQL para crear suscripciones, verificar Wompi y resolver pagos manuales.
- **lib/catalog.ts & lib/catalogDb.ts:** Catálogo unificado y consultas NoSQL con fallback a datos locales.
- **lib/data.ts:** Base de datos en memoria para modo autónomo (películas, series, sinopsis, preguntas frecuentes).
- **lib/findTitle.ts:** Búsqueda predictiva multicriterio (título, sinopsis, género, año de estreno).
- **lib/mail.ts:** Cliente Nodemailer con fallback a terminal en desarrollo si no hay servidor SMTP configurado.
- **lib/myListStore.ts & lib/myListSync.ts:** Sincronización híbrida cliente-servidor (LocalStorage + Redis).
- **lib/seo.ts:** Definición centralizada de metadatos, openGraph canónicos y generador de sitemaps.
- **lib/slugs.ts:** Normalizador de cadenas a slugs web URL-friendly (eliminación de tildes y caracteres especiales).

6. SISTEMA DE DISEÑO (DESIGN SYSTEM) Y EXPERIENCIA DE USUARIO (UI/UX)

El diseño de PopConly está cuidadosamente curado para ofrecer una experiencia estética cinematográfica 'Dark Premium', con fondos profundos para resaltar la colorimetría de los pósteres y carátulas de vídeo.

6.1. Tokens Cromáticos Corporativos (@theme en Tailwind CSS v4)

TOKEN	VALOR HEX	ROL EN EL SISTEMA DE DISEÑO
`--color-void`	#0D0D11	Fondo oscuro cinematográfico absoluto (Void). Reduce el cansancio ocular en salas oscuras.
`--color-surface`	#131317	Superficie de tarjetas primarias, modales de detalle y barras de navegación.
`--color-surface-container`	#1F1F23	Superficie elevada para inputs de formularios y contenedores de opciones secundarias.
`--color-primary-deep`	#FF2A4D / #E50914	Rojo vibrante corporativo. Botones de reproducción principal, badges de 'Estreno' y acentos del logotipo.
`--color-amber`	#FEB700	Dorado ámbar para calificaciones críticas (Match 98%), sellos de calidad 4K e insignias de verificación.
`--color-violet-container`	#917EFF	Acento violeta neón para resplandores ambientales y perfiles secundarios (Kids / DocuZone).
`--color-on-surface`	#E4E1E7	Tipografía principal de alto contraste y legibilidad optimizada sobre fondos negros.

6.2. Tipografía y Micro-animaciones Interactivas

• **Tipografía:** Se utiliza la familia tipográfica **Plus Jakarta Sans** cargada optimizadamente con next/font/google (pesos 400, 600, 700 y 800), combinada con el set de glifos vectoriales **Material Symbols Outlined**.
• **Micro-animación Like-pop:** Efecto elástico con keyframes CSS (@keyframes like-pop) que escala el icono a 145% y desciende a 90% con amortiguación suave al pulsar el botón de recomendación positiva.
• **Botón Morfogenético (Morph-btn Plus a Check):** Componente SVG en components/ActionButtons.tsx. Al ser pulsado, el signo más (+) rota 90 grados y se desvanece mientras la marca de verificación (✓) se dibuja dinámicamente mediante interpolación de la propiedad stroke-dashoffset de 100 a 0.
• **Buscador Instantáneo & Vista Previa:** Animación @keyframes preview-in con escalado sutil de 82% a 100% que muestra los resultados de búsqueda de forma reactiva sin recargar la página.

6.3. Componentes Reutilizables de Interfaz (Directorio components/)

• **BrowseHeader:** Barra de navegación con efecto blur al hacer scroll, buscador expansible, panel de notificaciones y selector de perfiles.
• **TitleModal:** Modal de detalle completo con reproducción automática de tráiler, lista de episodios, especificaciones técnicas (Dolby Vision, Atmos) y reparto.
• **PrototypeCards:** Tarjetas de catálogo con badges de estado, porcentajes de progreso y transiciones de zoom en hover.
• **Logo:** Isotipo vectorial con triángulo de reproducción, resplandor rojo y punto ámbar de estado.

7. MOTOR FINANCIERO (WOMPI + NEQUI) Y ESTRATEGIA SEO

PopConly implementa un modelo de facturación pensado especialmente para el mercado latinoamericano, permitiendo cobrar mediante pasarelas bancarias automatizadas o transferencias móviles sin intermediarios:

PLAN	PRECIO USD	PRECIO COP	CALIDAD	PANTALLAS	DESCARGAS
Pop Essential	$4.99 / mes	$19.900 COP	720p HD	1 simultánea	Sin descargas
Pop Plus (Popular)	$8.99 / mes	$36.900 COP	1080p Full HD	2 simultáneas	Habilitadas
Pop Ultra	$12.99 / mes	$52.900 COP	4K HDR + Atmos	4 simultáneas	Todos los disp.

Flujo de Tokenización con Wompi (Bancolombia): Para cumplir con la normativa PCI-DSS, el número de tarjeta y el CVC **nunca viajan a nuestro servidor**. En su lugar, el frontend (app/signup/payment) envía los datos directamente a la API de Wompi (https://sandbox.wompi.co/v1/tokens/cards) usando la llave pública. Wompi devuelve un token seguro con el cual nuestro backend en /api/checkout formaliza la transacción. Posteriormente, el webhook en /api/webhooks/wompi valida la firma criptográfica SHA-256 antes de actualizar el estado en MySQL a 'active'.
Flujo de Pago Manual con Nequi: Pensado para usuarios no bancarizados o pruebas académicas. El usuario transfiere al número indicado, registra el número de referencia y su nombre, y la solicitud queda en estado 'manual_review'. El administrador puede ingresar a /admin/subscriptions y aprobar el pago con un solo clic, habilitando el catálogo.

7.1. Estrategia de SEO Técnico y Semántica Web

• **Metadatos Canónicos y OpenGraph:** Cada ruta (/browse, /movies, /series, etc.) define títulos con plantilla, descripciones optimizadas y etiquetas OpenGraph / Twitter Cards para previsualización enriquecida en redes sociales.
• **Fichas de Título Indexables (/titulo/[slug]):** Cada película o serie cuenta con su propia URL permanente generada estáticamente con generateStaticParams() y metadatos dinámicos generados desde MongoDB o catálogo local.
• **Datos Estructurados JSON-LD (Schema.org):** Inyección en el DOM de esquemas oficiales: 'Organization' y 'WebSite' en el layout principal, y esquemas 'Movie' y 'TVSeries' con sinopsis, fecha de estreno, calificación por edad y géneros en cada título.
• **Rutas de Rastreo Automático:** Implementación de app/robots.ts (con reglas para GoogleBot) y app/sitemap.ts, que construye dinámicamente un mapa XML con todas las URLs del sitio y sus títulos actualizados.

8. INFRAESTRUCTURA, DEVOPS Y DESPLIEGUE EN ORACLE CLOUD (OCI)

El proyecto fue diseñado meticulosamente para alojarse de forma 100% gratuita dentro del nivel Always Free de Oracle Cloud Infrastructure (OCI), utilizando la máquina virtual Ampere A1 basada en arquitectura ARM64:

SERVICIO / CONTENEDOR	IMAGEN BASE	PUERTO LOCAL	VOLUMEN PERSISTENTE / CONFIGURACIÓN
postgres	`postgres:16-alpine`	5432	pgdata:/var/lib/postgresql/data (Autenticación y usuarios)
mongo	`mongo:7`	27017	mongodata:/data/db (Catálogo multimedia enriquecido)
redis	`redis:7-alpine`	6379	redisdata:/data (--maxmemory 1gb, LRU eviction)
mysql	`mysql:8.4`	3306	mysqldata:/var/lib/mysql (Ledger de cobros y Wompi)
app (Next.js)	`Dockerfile (Node 22)`	3000	Compilación Multi-Stage Standalone. Se comunica por red interna.

Optimización del Dockerfile (Compilación Multi-stage): Se estructura en 3 fases (deps, builder y runner). La fase final solo incluye el bundle standalone de Next.js y los estáticos de .next/static y public, reduciendo el tamaño de la imagen de más de 1.2 GB a menos de 180 MB.
Proxy Inverso y Certificados SSL Automáticos con Caddy: Delante del puerto 3000 se ubica Caddy Server en la VM. Con solo 3 líneas de configuración en su Caddyfile, gestiona automáticamente certificados HTTPS válidos de Let's Encrypt para el dominio popconly.tv, cumpliendo el requisito estricto de Wompi y Google para pagos y SEO.
Reglas de Red y Firewall: El despliegue requiere configurar las reglas de entrada (Ingress Rules) en la VCN de Oracle para los puertos 80 y 443 TCP, y guardarlas persistentemente en Linux mediante iptables y netfilter-persistent.

9. CONCLUSIONES Y HOJA DE RUTA RECOMENDADA

1. **Nivel de Madurez del Código:** El proyecto se encuentra en un estado excepcional de acabado visual y de arquitectura de software. Posee tipado estricto, micro-interacciones de alta fidelidad, modularidad de componentes y un cableado completo hacia bases de datos relacionales, documentales y en memoria.
2. **Ventaja del Modo Fallback:** La capacidad de funcionar de forma autónoma sin exigir bases de datos activas permite realizar demostraciones comerciales y pruebas de usabilidad de forma inmediata sin dependencias complejas.
3. **Pasos Recomendados para Lanzamiento a Producción:**
• Levantar los contenedores en Oracle Cloud: docker compose up -d.
• Ejecutar migraciones de Prisma (npx prisma migrate deploy) y el script de MongoDB (npm run db:seed:catalog).
• Reemplazar las credenciales sandbox de Wompi por llaves de producción y configurar el webhook oficial.
• Integrar un CDN de distribución de vídeo (como Cloudflare Stream o AWS CloudFront sobre OCI Object Storage) para servir archivos HLS adaptativos sin sobrecargar el ancho de banda de la máquina virtual.